

HE²: A Communication-Light Heterogeneous Architecture for Efficient Fully Homomorphic Encryption

Shangyi Shi^{1,2,3}, Husheng Han¹, Zhaoxuan Kan^{1,2,3}, Yinghao Yang¹, Jianan Mu¹,
Tenghui Hua¹, Ge Yu^{1,2,4}, Xinyao Zheng^{1,2,3}, Ling Liang⁵, Zidong Du^{1,2}, Xing Hu^{1,2}

¹State Key Laboratory of Processors, Institute of Computing Technology, CAS, Beijing, China

²University of Chinese Academy of Sciences, Beijing, China ³Cambricon Technologies

⁴School of Advanced Interdisciplinary Sciences, CAS, Beijing, China

⁵School of Integrated Circuits, Peking University, Beijing, China

{shishangyi22s, hanhusheng, kanzhaoxuan23z, yangyinghao}@ict.ac.cn

{mujianan, huatenghui24s, yuge23s, zhengxinyao22s}@ict.ac.cn

lingliang@pku.edu.cn {dudong, huxing}@ict.ac.cn

Abstract—CKKS, an emerging fully homomorphic encryption (FHE) scheme, has been promising in privacy-preserving applications by enabling SIMD fixed-point computations on ciphertexts. Despite its strong security guarantees, CKKS involves both compute-intensive operators (ComOps) with high computational cost and memory-intensive operators (MemOps) with large memory footprints, making existing ASIC-based or NMP-based acceleration approaches suffer from high hardware overhead and limited efficiency. This observation motivates the integration of the architectural advantages of both paradigms into a heterogeneous xPU (ASIC)-xMU (NMP) architecture. However, in such a design, frequent and long-latency heterogeneous communication caused by the dominant keyswitch operator remains a key performance bottleneck.

In this paper, we propose HE², a communication-light xPU-xMU heterogeneous FHE accelerator with dataflow graph (DFG) optimization and architecture co-design. First, we observe that the majority of communication arises at the interface between ModUp/ModDown and neighboring MemOps. To address this, we propose a DFG-level optimization framework to fully exploit the ModUp/ModDown reduction potential of the hoisting algorithm by identifying parallel keyswitch blocks and fusing them for reduced communication frequency. Second, we design an efficient heterogeneous architecture that adopts a group-level pipelined execution to effectively hide communication latency by leveraging the inherent parallelism across decomposed groups. End-to-end evaluation results show that HE² achieves 1.66 $\times$ speedup and 9.23 $\times$ lower EDAP (Energy-Delay-Area Product) compared to the state-of-the-art accelerator, with communication stalls accounting for only 6.67% of the total latency.

Index Terms—Fully Homomorphic Encryption (FHE), Heterogeneous Architecture, Dataflow Graph Optimization.

I. INTRODUCTION

Fully Homomorphic Encryption (FHE) has attracted significant attention due to its ability to enable computation on encrypted data in wide privacy-preserving cloud scenarios. Among existing FHE schemes [3], [7], [8], CKKS stands out

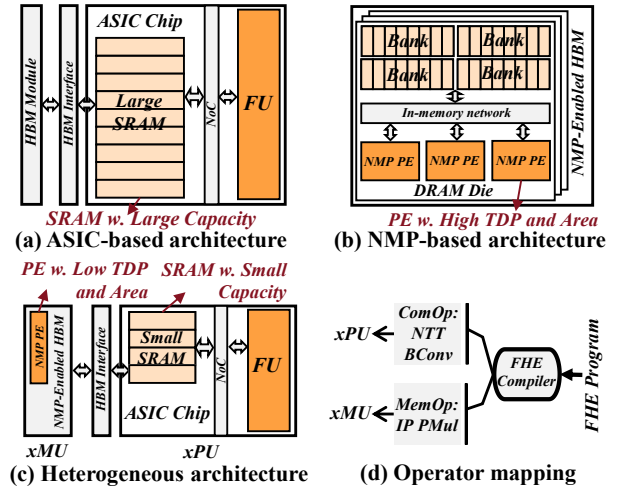


Fig. 1. Both ASIC and NMP architectures suffer from high hardware overhead due to large on-chip SRAM (a) and near-memory compute integration (b); the heterogeneous design exploits the strengths of both computation and memory access to achieve a more practical trade-off (c), by mapping ComOps to the ASIC side and MemOps to the NMP side (d).

for supporting approximate arithmetic and efficient SIMD-style computations, making it particularly suitable for privacy-preserving machine learning [5], [21], [30], [31], [53].

CKKS is characterized by both computation-intensive operators (ComOps) with complex computation patterns like ModUp and ModDown, and memory-intensive operators (MemOps) like Inner Product (IP) and Plaintext Multiplication (PMul). Prior works have explored two monolithic accelerator paradigms. First, the ASIC designs [2], [12], [25], [26], [43], [44], [55] employ customized modules to accelerate complex ComOps, but require large on-chip memory to alleviate the memory bottleneck, resulting in substantial area and power costs (Fig. 1(a)). Second, the Near-Memory Processing (NMP)

Corresponding author is Husheng Han.

designs [14], [40], [54] leverage high near-memory bandwidth for MemOps but require integrating large computing cores for ComOps (Fig. 1(b)). The integration of a massive complex logic circuit incurs substantial area and power overheads, which hinder its feasibility for implementation [23], [32]. Both design methodologies face the dilemma of efficiency and hardware costs.

The heterogeneous ASIC-NMP architecture (Fig. 1(c)) is a promising solution to exploit the complementary strengths of both approaches: high-performance ASIC-based modules (xPU) accelerate ComOps, and high-bandwidth NMP modules (xMU) alleviate memory bottlenecks for MemOps.

However, *a key challenge of the heterogeneous architecture is the significant communication overhead between the xPU and xMU* due to the computation pattern of the core operation keyswitch (occupies 80% computation in CKKS). The dataflow of keyswitch consists of an alternating sequence of ComOps and MemOps (ModUp-IP-ModDown), leading to **① frequent communication when ComOps are mapped onto the xPU and MemOps onto the xMU**. In addition, the data exchanged between ComOps and MemOps—ciphertexts output from ModUp and IP—are **② large in size, resulting in considerable single-transfer latency**. Unlike monolithic ASIC designs, where off-chip data can be preloaded to hide latency, the communication for the on-the-fly intermediate results in the heterogeneous architecture lies on the critical path, posing a fundamental bottleneck. In our evaluation, a heterogeneous accelerator, combining a state-of-the-art monolithic xPU [25] with bank-level PE-integrated xMU via a 1 TB/s HBM interface, achieves $1.11\times$ reduction in the bootstrapping computation latency but incurs $8.65\times$ increase in communication stalls.

In this paper, we propose a communication-light heterogeneous CKKS accelerator with dataflow graph (DFG) optimization and architecture co-design. To address the frequent heterogeneous communication, we propose **HERO**, a DFG optimization framework to *maximize the communication-reduction benefits of hoisting*. The hoisting algorithm [4] reduces ComOps and communication in heterogeneous systems. Nonetheless, its benefit is constrained by the limited keyswitch parallelism in the CKKS DFG. To circumvent this bottleneck, HERO identifies parallel keyswitch blocks, expands and fuses them to enhance parallelism and maximize hoisting’s communication-reduction potential.

To alleviate the on-critical-path communication stall, we propose **HE²**, an heterogeneous FHE architecture, *utilizing the group-granularity pipeline design to hide the communication latency and improve computation efficiency*. For the xPU, we design dual-level pipelined computation modules to enable both computation-communication overlap and inter-operator overlap for efficient communication hiding. For the xMU, we design highly parallel yet lightweight modules that exploit the high bank-level bandwidth of HBM.

Our main contributions are as follows:

- We construct the first xPU (ASIC)-xMU (NMP) heterogeneous architecture to fully exploit the efficiencies of

TABLE I
ARITHMETIC INTENSITY (AI) (OPS PER BYTE) OF CKKS OPERATORS.
EVALUATED UNDER THE SHARP [25]’S PARAMETERS.

	Compute Intensive Ops				Memory Intensive Ops			
Op	(I)NTT	BConv	ModUp	ModDown	IP	PMul	CAdd	Rescale
AI	0.89	1.60	3.38	2.92	0.12	0.09	0.07	0.11

the xPU for complex ComOps and the high-bandwidth xMU for MemOps, and identify that the bottleneck lies in the intermediate results communication between ComOps and MemOps with detailed profiling.

- To address the heavy communication, we propose a co-design that includes a DFG optimization framework for communication-reduction and an inter-group dual-level pipelined hardware for communication-hiding.
- Compared to the state-of-the-art CKKS accelerator SHARP [25], end-to-end evaluation results show that HE² achieves $1.66\times$ speedup and $9.23\times$ EDAP improvement. Specifically, DFG optimization enables hoisting to reduce computation workload and communication volume by an average of $1.64\times$ and $3.27\times$, and the pipelined hardware reduces communication stalls to 6.67%.

II. BACKGROUND

A. Basic operators

CKKS is composed of fundamental polynomial-level operators, which are the primary module for acceleration, including computation-intensive (ComOps) and memory-intensive operators (MemOps), with arithmetic intensity shown in Table I.

NTT (Number-Theoretic Transform) transforms an input polynomial of degree N from the coefficient to the slot domain through $O(N \log N)$ butterfly operations.

BConv (Basis Conversion) transforms polynomials within one RNS basis into another. By performing a constant multiplication under the original basis with l_1 moduli, followed by another constant multiplication and reduction under l_2 target moduli bases, BConv incurs a complexity of $O(l_1 \cdot l_2 \cdot N)$.

IP (Inner Product) involves multiplying a group of polynomials with two groups of evk polynomials, followed by a sum reduction. IP incurs a substantial memory footprint and a small computation intensity.

EWO (Element-Wise Operation) encompasses operations such as ciphertext-ciphertext addition (CAdd), plaintext-ciphertext addition (PAdd), and multiplication (PMul). EWO exhibits low computation intensity and only involves element-wise addition or multiplication.

Autom (Automorphism) permutes the polynomial, with each coefficient index i mapped to $ik \bmod N$.

B. Critical primitive in CKKS

1) *keyswitch*: In keyswitch [24], the ciphertext under modulus Q is decomposed into $dnum$ groups, lifted to $PQ \cdot dnum$ via **ModUp**, multiplied with the evk (**IP**), and reduced back

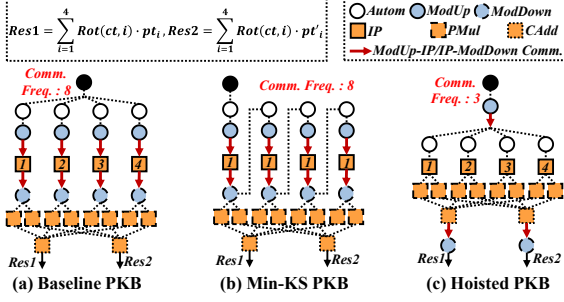


Fig. 2. The original PKB performs eight ComOp-MemOp communications (a). Min-KS reduces the number of *evk* but not the communication frequency (b). Hoisting removes redundant ModUps and ModDowns, lowering the ModUp→IP and IP→ModDown communications, respectively (c).

to Q by **ModDown**. This result is added to the original ciphertext to complete the keyswitch. In CKKS, both ciphertext multiplication and rotation depend on this core primitive.

2) *Modulus-Commutative Property*: All EWOs and Autom can exchange their execution order with ModUp and ModDown [1], [4], [27]. We denote such operators as **Commutative Operators**. Taking PMul and CAdd as examples, for ciphertexts ct , ct' , and plaintext pt , the following Equations (1) and (2) hold. Note that in the right-hand side of Equation (1), the plaintext must be lifted via PModUp as described in [1] to stay in the same domain as ModUp(ct).

$$\text{ModUp}(\text{PMul}(ct, pt)) = \text{PMul}(\text{ModUp}(ct), \text{PModUp}(pt)) \quad (1)$$

$$\text{ModUp}(\text{CAdd}(ct, ct')) = \text{CAdd}(\text{ModUp}(ct), \text{ModUp}(ct')) \quad (2)$$

C. Algorithmic Optimizations for PKB

Multiplication between a plaintext matrix and a ciphertext vector is the key computation in machine learning and C2S/S2C of bootstrapping [7]. In state-of-the-art implementations [6], [22], it uses parallel keyswitches with varying rotation steps, followed by linear combinations via PMul and CAdd. We define blocks of parallel keyswitches as PKBs, as shown in Fig. 2(a). In plaintext-matrix-ciphertext multiplication, the rotation steps in PKB form an arithmetic progression.

Baby-Step Giant-Step (BSGS) [22], [26] algorithm divides the PKB into groups of smaller PKBs shown in Equation (3) and reduces both the computational complexity and the number of *evk* from $O(n_1 \cdot n_2)$ to $O(n_1 + n_2)$.

$$\begin{aligned} & \sum_{i=1}^{n_1 \cdot n_2} \text{PMul}(\text{Rot}(ct, s_i), pt_i) \\ &= \underbrace{\sum_{i=1}^{n_1} \text{Rot}\left(\sum_{j=1}^{n_2} \text{PMul}(\text{Rot}(ct, s_j), pt_{i,j}), s_i\right)}_{\text{Baby step}} \quad (3) \end{aligned}$$

The **Min-KS** [26] approach further reduces the required *evk* number by converting the PKB to serial rotations to ensure that each rotation step is uniform (Fig. 2(b)).

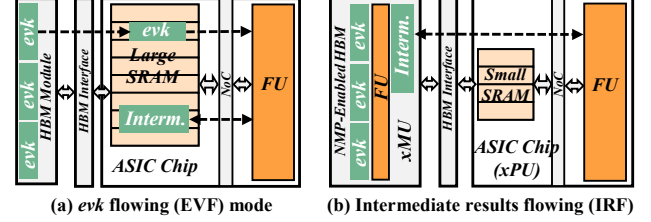


Fig. 3. Two dataflow schemes exist in an ASIC chip equipped with HBM. An *evk* flowing dataflow preloads *evk* to the chip for IP computation (a). An intermediate results flowing scheme transfers intermediates of keyswitch to xMU for IP computation, where data transfers are on the critical path (b).

The **hoisting** technique [4] exchanges ModUps/ModDowns with commutative operators and merges those from different keyswitches, as shown in Fig. 2(c). By reducing the number of ComOps, hoisting lowers communication frequency between ComOps and MemOps. However, while it lessens the ComOps workload, it increases that of MemOps, whose computation order is swapped, shifting their modulus domain from Q to PQ or $PQ \cdot dnum$.

III. MOTIVATION

A. Existing ASIC/NMP-based Architectures Face Dilemma of Performance and Hardware Costs

The coexistence of compute-intensive operators (ComOps) and memory-intensive operators (MemOps) in CKKS poses major challenges for hardware-efficient acceleration. Prior studies have explored ASIC-based [2], [12], [25], [26], [29], [36], [43], [44] and NMP-based [14], [40], [54] monolithic architectures, both incurring substantial hardware overhead. **First**, ASIC accelerators achieve efficient ComOps acceleration with low area and power consumption by using advanced process nodes [9] and employ customized computation modules. However, CKKS workloads demand large on-chip SRAM for MemOps, which can consume up to half of the total area and power consumption in ASIC accelerators [26], [29]. **Second**, NMP designs exploit high near-memory bandwidth to accelerate MemOps and integrate deeply pipelined compute units within low hierarchies in DRAM for ComOps [40], [45], but this results in complex and costly designs. Specifically, pure in-die NMP cannot provide adequate computational capability for FHE within the area budget (25% suggested by SK Hynix AiM [23], [32]), owing to the scarcity of logic resources in DRAM's technology that transistors operate roughly $3\times$ slower and the density is $10\times$ lower than CMOS at the same technology node [13]. Additionally, the support for complex FHE ComOps leads to considerable power overhead, necessitating customized high-end-server active heat sink thermal management [20], [40], thereby further complicating industrial tape-out. These two paradigms inspire us to propose a hardware-efficient ASIC (xPU)-NMP (xMU) heterogeneous architecture featuring an xPU with small on-chip memory and an xMU supporting only simple MemOps, which introduces low extra logic integration.

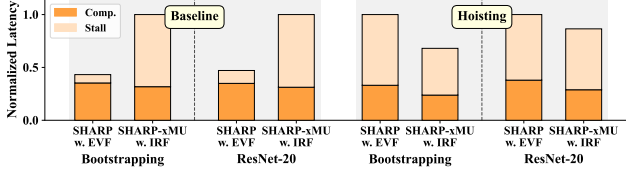


Fig. 4. Baseline adopts Min-KS as in [25], [26]. SHARP-xMU is an xPU-xMU heterogeneous accelerator, in which the xPU is aligned with SHARP with detailed configurations provided in Sec. VI-A. Communication is the primary bottleneck in IRF-based heterogeneous architectures, and hoisting can partially reduce communication overhead.

B. Frequent and Large Communication Traffic is the Key Challenge in Heterogeneous Architectures

According to prior ASIC designs [25], [29], large on-chip SRAM stores preloaded *evks* for IP and baby-step ciphertexts for PMul in the BSGS phase of bootstrapping. To lower the on-xPU memory demand, an efficient mapping of IPs and PMuls is essential.

IP appears exclusively in the keyswitch. In an xPU-xMU heterogeneous system, keyswitch admits two dataflow design choices: the *evk flowing* (EVF) dataflow used in existing ASIC monolithic accelerators (Fig. 3(a)) and a new *intermediate results flowing* (IRF) dataflow (Fig. 3 (b)). The EVF dataflow offloads the entire keyswitch to the xPU. It allows *evk* to be reused and preloaded to reduce the cost of off-chip memory access. However, when the program exhibits sequential keyswitches with low *evk* reuse, its performance remains constrained by large on-xPU memory [26]. The IRF dataflow maps IPs to the xMU to avoid loading *evk* to the xPU. In this mode, the ModUp output is sent to the xMU for IP, and the result is then returned to the xPU for ModDown.

Although IRF reduces xPU memory use and exploits xMU near-memory bandwidth, it introduces two additional data transfers per keyswitch, i.e., between ModUp IP and IP ModDown. Considering the dominance of keyswitch in CKKS, **1 the communication between xPU and xMU becomes frequent and degrades the overall performance when IRF is applied.** Furthermore, this communication lies on the critical path, involving up to 144 MB of intermediate ciphertext per transfer. We extend a state-of-the-art ASIC design [25] by integrating an xMU component into a heterogeneous xPU-xMU architecture. In this architecture, **2 the existing xPU design cannot effectively balance communication and computation, making it difficult to hide the latency of intermediate result transfers along the critical path.** In our experiments, this communication stall accounts for 68.2% and 68.7% of the total latency in bootstrapping [6] and ResNet-20 [30], as shown in the left region of Fig. 4.

C. Communication Optimization Potential of Hoisting and Its Limitation under Low Keyswitch Parallelism

Hoisting [4] extracts shared ModUps and ModDowns from multiple parallel keyswitches. Although the corresponding *evks* differ, the ModUp results can be reused across IPs

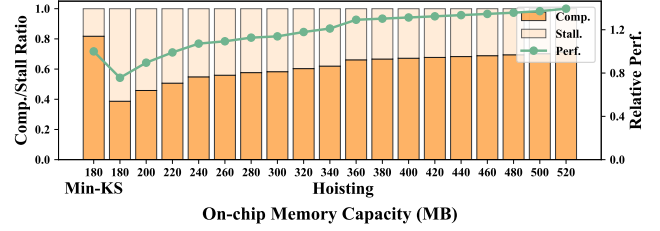


Fig. 5. Bootstrapping performance of SHARP with hoisting across varying on-chip memory capacities. Increasing on-chip memory allows *evk* preloading to mitigate off-chip stalls, but reduced *evk* reuse under hoisting yields diminishing performance gains.

after hoisting, and the aggregated results from multiple IPs require only one ModDown. Thus, hoisting trades *evk* reuse for improved reuse of intermediate results.

Due to reduced *evk* reusability, hoisting causes memory access stalls and performance loss in EVF-based architectures. As depicted in Fig. 5, directly applying hoisting to SHARP leads to performance degradation and offers only a 39.4% speedup despite a $2.89\times$ increase in on-chip memory. In contrast, **hoisting improves intermediate result reuse, reducing communication frequency in IRF-based heterogeneous architectures and thus boosting performance.** Consequently, the IRF-based system achieves moderately higher performance than its EVF-based counterpart when hoisting is applied, as shown in the right region of Fig. 4. Moreover, hoisting enables PMul in BSGS of the bootstrapping to be reordered and offloaded to the xMU along with adjacent IPs (Fig. 2). Consequently, the substantial on-chip memory footprint of baby-step ciphertexts described in Sec. III-B can be avoided. For bootstrapping and ResNet-20, hoisting can reduce the communication stall of IRF-based systems by $2.00\times$ and $1.61\times$ compared to the two baseline benchmarks, respectively.

Nevertheless, **3 the optimization effect of hoisting on communication is limited due to the low keyswitch parallelism of CKKS programs.** Specifically, through hoisting, redundant ModUps and ModDowns within the PKB can be reduced to a level proportional to the PKB's input and output degrees. Since the saved communication volume scales with the reduction in ModUp and ModDown, a PKB with higher parallel keyswitches and lower in/out-degree can achieve more communication savings. In CKKS programs, many fragmented PKBs with a limited keyswitch-parallelism (less than 10) exist, as shown in Fig. 6, which restricts the hoisting effect. Therefore, in Sec. IV, we propose a program graph level optimization method (HERO) to fuse multiple low-parallelism PKBs into a few PKBs with higher parallelism (more than 30), to fully exploit the potential of hoisting.

D. New Heterogeneous-Specific Performance Model is Needed

Since this work marks the first attempt at an ASIC-NMP heterogeneous acceleration for CKKS, and prior studies have not thoroughly explored the potential of hoisting, we observe that the performance models for existing monolithic CKKS accelerators, which are primarily based on computation volume

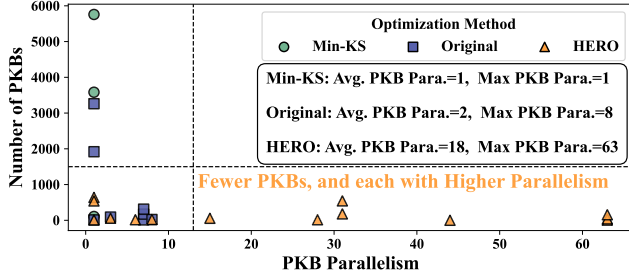


Fig. 6. Parallelism and number of PKBs in the bootstrapping, ResNet20, and HELR. CKKS program contains numerous segments with low keyswitch parallelism. Min-KS further increases the number of PKBs while producing PKBs with lower parallelism. By applying our algorithmic optimization introduced in Sec. IV, low-parallelism PKBs are fused into a smaller number of highly parallel PKBs, thereby unveiling the benefits of hoisting.

and off-chip memory access stall under the EVF dataflow, are insufficient to accurately assess the impact of hoisting and IRF in heterogeneous systems. In prior work on BSGS of bootstrapping with EVF dataflow, the baby-step (*bs*) and giant-step (*gs*) are jointly optimized to minimize computation cost and off-chip memory traffic. The computation cost is lowest when *bs* and *gs* are equal (i.e., both 8), but this exceeds the on-chip memory for baby-step ciphertexts [25], causing frequent intermediate swaps. Thus, the optimal configuration of [25] adopts a smaller *bs* of 4 (Fig. 7(a)).

In contrast, with hoisting and IRF, choosing larger gaps between *bs* and *gs* exposes more parallelism for hoisting, reducing communication and computation. However, such configurations increase the *evk* storage demand and may exceed HBM capacity, as depicted in Fig. 7(b). This motivates the need to **develop a new heterogeneous-specific performance model that jointly considers computation, communication, and the size of *evk* working set for general CKKS applications.**

IV. HERO: A HOISTING-ENHANCED DFG OPTIMIZATION FRAMEWORK

To tackle the critical communication challenges on heterogeneous systems, and to explore the optimization potential of hoisting, we abstract dataflow graphs (DFGs) from applications, where nodes represent operators and edges denote data dependencies. Based on this, we propose **HERO**, a **hoisting-enhanced communication reduction DFG optimization framework**. The input to HERO is a DFG generated by an FHE compiler [10], [11], [35]. To enable the generality of hoisting in arbitrary CKKS programs, we identify and locally expand PKBs for degrees fine-tuning (Sec. IV-A). This preprocessing stage reshapes the local DFG, thereby enhancing the effectiveness of hoisting on subgraphs. Furthermore, to enhance the keyswitch-level parallelism that hoisting can leverage, we introduce, for the first time, a novel technique called PKB fusing, which fuses PKBs at the global DFG level and leverages a fusion evaluator to derive the optimal fusion scheme (Sec. IV-B). Finally, we map the restructured DFG onto the proposed heterogeneous accelerator based on the parallelism of each PKB (Sec. IV-D).

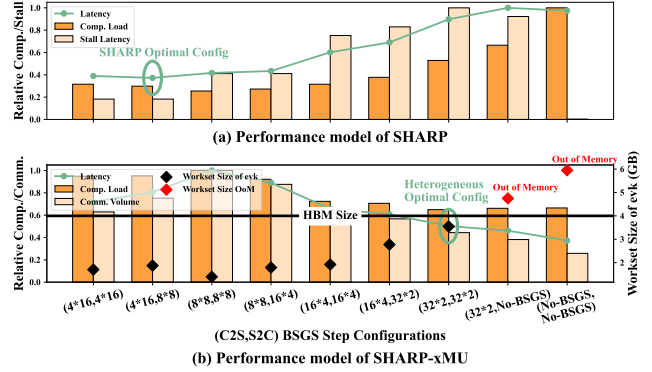


Fig. 7. BSGS parameter exploration. EVF-based monolithic architectures (e.g., SHARP) consider computation load and off-chip memory access stalls. In contrast, the performance model of IRF-based heterogeneous accelerators (e.g., SHARP-xMU) incorporates computation load, communication volume, and the *evk* workset size. The difference in the considered factors leads to distinct optimal parameter configurations.

A. Preprocessing: Subgraph Degrees Minimizing for Locally Optimal Hoisting

Existing hoisting-based studies [1], [16], [27] achieve better hoisting effects by adjusting commutative operators to reduce the degrees of PKBs during the bootstrapping. Our preprocessing procedure performs similarly, but it is not limited to a certain application [15]. Instead, we propose a general method by formalizing this process into the following two steps, making it applicable to PKBs within arbitrary CKKS programs, as illustrated in Fig. 8(a) and (b).

PKB identifying. We traverse the DFG from the inputs, assign keyswitches to layers based on their order along each path, and group those in the same layer into PKBs, yielding a partitioned DFG composed of sequential PKB layers and intermediate operations, as shown in Fig. 8(a).

Degree-minimized PKB expanding. Hoisting places the ModUp and ModDown of a PKB at its input and output, respectively. To maximize its benefit (i.e., reduce the number of ModUps and ModDowns), we minimize each PKB's in- and out-degree by greedily expanding it with commutative operators, and apply hoisting as shown in Fig. 8(b) and (c).

As noted in Sec. II-B2, this expansion enlarges the modulus domain and thus increases the cost of MemOps (e.g., PMul, CAdd), but the overhead is generally outweighed by the reduced ModUps and ModDowns.

B. PKB Fusing for Hoisting Efficiency Enhancement

Although each PKB is locally optimized through subgraph degrees minimizing, the parallelism of keyswitch within CKKS programs remains unchanged and low (Fig. 6), which limits the overall gains of hoisting.

Unlike prior works [1], [16], [27] that apply hoisting without modifying the underlying CKKS program, we further enhance its benefits by proposing **PKB Fusion**, which enlarges the parallelism and reduces the in- and out-degree of PKBs. Specifically, consider two adjacent PKBs with n_1 and n_2

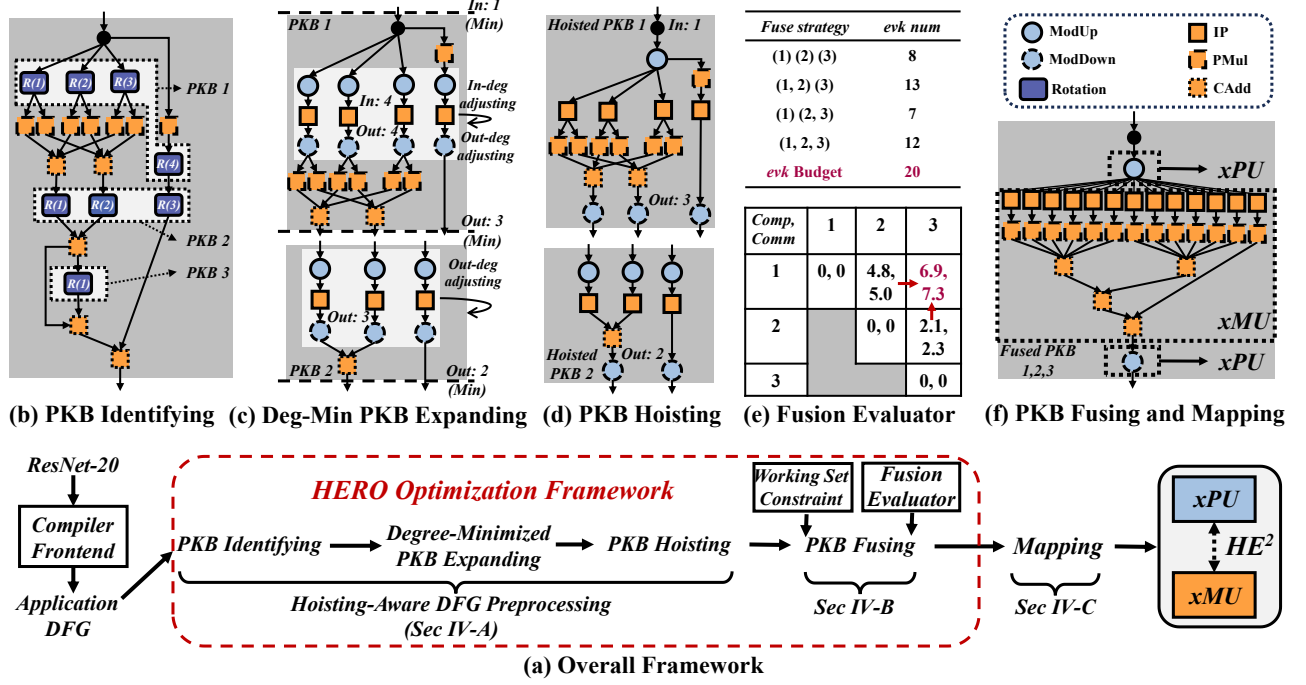


Fig. 8. The overall procedure of HERO (a). The PKB in CKKS DFG is first identified (b), followed by expanding each PKB for minimizing the degree (c) to enhance ComOp-MemOp communication reduction by hoisting and placing redundant ModUps and ModDowns to positions with minimized degrees (d). The fusion evaluator assesses the computation and communication reduction from fusing hoisted PKBs according to the relative MMul numbers and intermediates' size, along with the required *evk* counts (e), ultimately selecting the optimal fusing strategy under the storage capacity constraints (f). Identical indices in different rotation nodes do not imply the same rotation step size.

rotations, separated by EWOs F_i , where each rotation in the second PKB depends on all rotations in the first. We perform a transformation analogous to the inverse BSGS process (Equation (4)). Since EWOs such as PMul can commute with rotations, i.e., $\text{Rot}(\text{PMul}(ct, pt)) = \text{PMul}(\text{Rot}(ct), \text{Autom}(pt))$, we traverse each of the n_2 paths in the second PKB backward and push EWOs after the rotations. This makes each of the n_2 paths directly adjacent to one of the n_1 paths in the first PKB. We then exploit the additive property of rotations, i.e., $\text{Rot}(\text{Rot}(ct, s), t) = \text{Rot}(ct, s + t)$, to merge the two consecutive rotations into a single one. After pairing every rotation path in the second PKB with all n_1 paths in the first PKB, the two serial PKBs are fused into a larger PKB with $O(n_1 \cdot n_2)$ rotations, where the output nodes of PKB1 and input nodes of PKB2 are removed. Consequently, hoisting on this fused PKB eliminates additional ComOps and yields more savings in both computation and communication.

$$\begin{aligned}
 & \underbrace{\{\text{Rot}(F_i(\{\text{Rot}(ct, s_j)\}_{1 \leq j \leq n_2}), s'_i)\}_{1 \leq i \leq n_1}}_{\text{The second PKB}} \\
 &= \underbrace{\{F'_i(\{\text{Rot}(\text{Rot}(ct, s_j), s'_i)\}_{1 \leq j \leq n_2})\}_{1 \leq i \leq n_1}}_{\text{The first PKB}} \\
 &= \underbrace{\{F'_i(\{\text{Rot}(ct, s_j + s'_i)\}_{1 \leq j \leq n_2})\}_{1 \leq i \leq n_1}}_{\text{The Enlarged PKB}}
 \end{aligned} \tag{4}$$

However, this comes with a trade-off: while hoisting be-

comes more beneficial, the number of *evks* increases, the amount of IPs grows, and additional MemOp overheads are introduced. Therefore, **when applying PKB fusing, it is necessary to balance computation cost, communication overhead, and the working set size of evks.**

Formalization. The impact of PKB fusion can be modeled as follows. We consider two PKBs to be fused: PKB1 with out-degree $outdeg_1$ and n_1 rotations, and PKB2 with in-degree $indeg_2$ and n_2 rotations. After fusion, on the one hand, hoisting eliminates extra $outdeg_1$ ModDowns and $indeg_2$ ModUps, along with the associated communication volume. On the other hand, fusion introduces additional (1) up to $n_1 \cdot n_2 - n_1 - n_2$ IPs and *evk* storage overhead, with the actual number determined by the non-duplicated subset among $n_1 \cdot n_2$ keys, (2) the extra computation incurred by increasing the modulus of the EWOs between two PKBs.

Case Study. We illustrate the working mechanism and effects of PKB fusion through a simplified example of the ConvBN DFG in [30]. Fig. 9(a) shows a DFG containing three PKBs, each with 9, 8, and 8 parallel rotations, respectively. In this configuration, the number of *evks* corresponds to the non-duplicated subset of 25 keys, with both the numbers of ModUps and ModDowns being 25. Fig. 9(b) demonstrates the effect of applying hoisting to the raw DFG, as done in Anaheim [27] and FAST [16]. In this case, the parallel ModUps in PKB1 are reduced from 9 to 1, and the ModDowns

can be extracted after linear combination and reduced from 9 to 8. However, for PKB2 and PKB3, hoisting does not provide further optimization, as the parallel rotations in these PKBs lack common predecessor or successor nodes, thereby preventing any reduction in ModUps/ModDowns via hoisting. In this case, because hoisting has no impact on the number of IPs, the number of *evks* is changed.

In Fig. 9(c), PKB2 and PKB3 are fused into an enlarged PKB. During fusion, all data-dependent paths are combined according to the additive property of rotation steps, as formulated in Equation 4, where n_1 and n_2 represent the numbers of data-dependent paths. Specifically, PKB2-1 and PKB3-1 contain dependent paths. In PKB2-1, two parallel paths are summed to produce the input to PKB3-1, which is subsequently processed through its own two parallel paths. When PKB2-1 and PKB3-1 are fused, their respective two parallel paths are combined via rotation-step addition, resulting in a fused PKB comprising four parallel paths, three of which correspond to parallel rotations. After fusion, the $n_1 + n_2$ rotations described in Equation (4) become $O(n_1 \cdot n_2)$ rotations, leading to an increased PKB parallelism. In this case, by applying hoisting as shown in Fig. 9(d), the high-parallel ModUps and ModDowns can be reduced and extracted to the front and end of the eight parallel paths of the fused PKB, without performing additional ModUps and ModDowns between PKB2 and PKB3 as in Fig. 9(b).

This example demonstrates that the combination of PKB fusion and hoisting effectively reduces the number of ModUps and ModDowns at the cost of increased *evk* working set and MemOp computation volume.

Fusion Evaluator. To evaluate the benefit of fusing under storage constraints, we define $FuseScore(i, j)$ as follows: if the number of *evks* after fusing PKB i and PKB j exceeds the available storage capacity, the score is marked invalid; otherwise, for PKB i with n_i rotations and PKB j with n_j rotations, $FuseScore$ denotes the maximal combined computation and communication savings among all consistent PKB pairs with an invariant product $n_i \cdot n_j$.

$$DP[i][j] = \max_{1 \leq j' \leq j-1} DP[i][j'] + DP[j' + 1][j] + FuseScore(j', j' + 1) \quad (5)$$

We formulate the global DFG fusion as a dynamic programming (DP) problem. A two-dimensional DP table is constructed, where diagonal entries represent the base case of unfused PKBs and are initialized to zero. Each entry (i, j) stores the optimal cumulative $FuseScore$ for covering PKB i through PKB j . The table is populated iteratively toward the upper-right corner according to the transition equation (5), and the resulting solution yields the globally optimal fusion plan that minimizes computation and communication overhead while satisfying storage constraints. Ultimately, compared with applying hoisting in unmodified programs, HERO achieves a $2.25\times$ more reduction in computation load and $2.42\times$ more reduction in communication volume. Detailed evaluation is in Sec. VII-C.

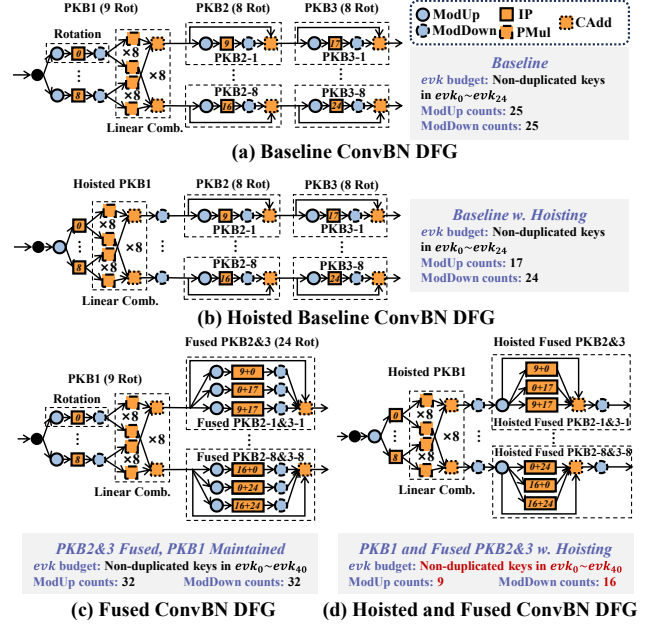


Fig. 9. PKB fusion example. The index inside the IP node represents a schematic value of the rotation step, rather than the actual step in real cases. As the number of fused PKBs increases, the fused PKB contains more parallel rotations, which requires a larger number of *evks* and shows more ComOps reduction with hoisting.

C. BSGS Configuration Exploration

The Baby-Step Giant-Step (BSGS) algorithm is a key technique to reduce the computational cost in the C2S and S2C phases of bootstrapping. C2S/S2C involves a PKB with D parallel rotations, followed by PMuls and CAdds. BSGS restructures this computation into two stages:

- **Baby step:** A ciphertext is rotated by bs different strides to obtain bs results, which are linearly combined via PMul and CAdd to yield $gs := D/bs$ results.
- **Giant step:** Each of the gs ciphertexts is further rotated once and summed to yield the final result.

Essentially, BSGS replaces one PKB with parallelism D and in/out-degree of 1 into two serial PKBs: **PKB1** with parallelism bs , in-degree 1, and out-degree bs ; and **PKB2** with parallelism gs , in-degree gs , and out-degree 1. Thus, **BSGS decreases overall keyswitch parallelism and increases the in/out-degrees of subgraphs**. When combined with hoisting, PKB1 has one ModUp and bs ModDowns, and PKB2 has gs ModUps and one ModDown. Without BSGS, only one ModUp and one ModDown are required. Although the number of IPs grows from $bs + gs$ to $O(bs \cdot gs)$, the total overhead tends to decrease.

In practical applications, HERO selectively disables BSGS when memory suffices for the extra *evk*. When the memory is limited, HERO prefers configurations with a larger gap between bs and gs , as this can lead to greater reductions in computation and communication overhead with hoisting, as shown in Fig. 7(b).

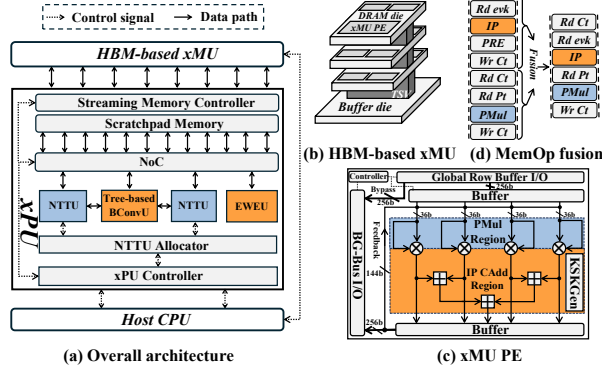


Fig. 10. The overall heterogeneous architecture contains an ASIC-based xPU (a) and an HBM-based xMU (b). The xMU PEs are deployed within the column decoder of all HBM banks (c). MemOps performed in xMU are fused to reduce write-back latency (d).

D. Dataflow Mapping

For the HERO-optimized program, its DFG is partitioned into a sequence of PKBs, each starting with ComOps (ModUps), followed by MemOps (IP, PMul, etc.), and ending with ComOps (ModDowns). In addition, EWOs may occur between adjacent PKBs. We consider two alternative mapping schemes. **IRF scheme.** We directly map ComOps and MemOps to the xPU and xMU, respectively, and map inter-PKB EWOs onto the xPU. In this case, all IPs are entirely executed on the xMU, allowing us to minimize on-xPU storage.

Hybrid scheme. For each PKB, we choose the dataflow based on its IP-level parallelism: we adopt IRF only when the IP parallelism is greater than 1; otherwise, we adopt EVF dataflow. The hybrid scheme enables us to use EVF when the cost of moving *evks* is lower than that of transferring intermediate results, thereby reducing communication overhead. However, it requires reserving additional on-xPU storage for one *evk*. We evaluate both schemes in Sec. VII.

V. HETEROGENEOUS ARCHITECTURE DESIGN

A. Architecture Overview

Although HERO reduces the frequency of heterogeneous communications, the latency of transferring intermediate results during keyswitch remains a bottleneck. The limitation arises from the xPU architecture’s design philosophy [16], [25], [26], [29], [43], which primarily prioritizes minimizing per-operator latency over balancing overall computation and communication delays. Consequently, the existing microarchitecture fails to simultaneously hide communication latency and sustain high overall performance.

We propose HE², a low-cost heterogeneous accelerator that delivers high performance and efficiently hides communication stalls. HE² consists of two main core components, as illustrated in Fig. 10.

xPU Design. We redesign the xPU to reduce per-operator hardware cost and balance operator latency with communication delay. Under limited single-operator performance, we

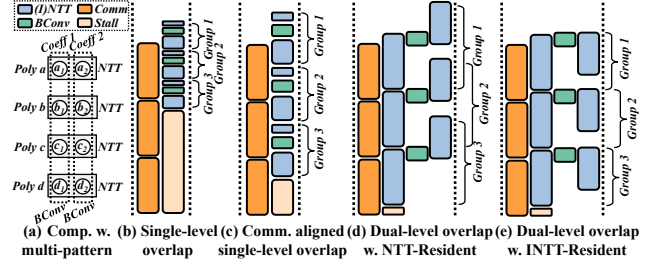


Fig. 11. Distinct computation patterns of NTT and BConv prevent efficient overlap (a). In a ModUp pipeline with 3 groups, the previous architecture [25] shows low overlap for both computation and communication (b). Naïvely reducing its parallelism partially mitigates communication stalls but at the cost of performance degradation (c). Dual-level overlapped xPU architecture hides substantial communication stalls (d), and the design of INTT-Resident keyswitch pipeline avoids performance degradation (e).

develop new microarchitectures to enable inter-operator overlap (Sec. V-B). Furthermore, we reconstruct the critical path from INTT→BConv→NTT into parallel BConv→NTT and NTT paths, improving computational parallelism (Sec. V-D). **xMU Design.** We propose a low-overhead and efficient xMU design along with the MemOps fusion technique and an in-memory automorphism method (Sec. V-E).

B. Dual-level Pipelining xPU Microarchitecture for Communication-Stall Mitigation

Compared with the monolithic architecture adopting a consistent EVF dataflow, HE² switches into an IRF dataflow for hoisted PKBs, where data transfers lie on the critical path of the keyswitch, exposing long communication stalls. Since MemOps executed in the near-memory module occupy the HBM ports and block concurrent xPU-HBM transfers and MemOps computation, we overlap communication and the computation on xPU, typically ModUp and ModDown.

ModUp and ModDown process multiple decomposed ciphertext groups, each following an INTT→BConv→NTT pipeline. Inter-group parallelism allows naïve computation communication overlap, where data transfers for one group are pipelined with computations for others. However, existing ASICs [2], [25], [26], [29] stack highly parallel and low-latency compute units, thus suffer from communication bottlenecks and underutilized compute resources, as shown in Fig. 11(b). Simply lowering computation parallelism cannot remove these stalls, as depicted in Fig. 11(c). To overcome this, we design an xPU with **dual-level overlapping**, combining computation-communication and inter-operator overlap to sustain acceleration performance while hiding communication latency.

Inter-operator overlap between (I)NTT and BConv is challenging because (I)NTT accesses different coefficients within one polynomial, whereas BConv processes coefficients from multiple polynomials concurrently, as depicted in Fig. 11(a). Existing designs [25], [26], [43], [44] offer high intra-polynomial but limited inter-polynomial parallelism in the NTTU, making its output throughput across limbs insufficient

TABLE II
MODELING CONFIGURATIONS OF HE² AND PRIOR WORKS.

Configs	SHARP	FAST	FHENDI	Anaheim xMU	HE ² -SM/LM xPU xMU
Word Width	36-bits	36/60-bits	46/51-bits	28-bits	36-bits
Core Freq.	1 GHz	1 GHz	-	0.38 GHz	1 GHz 0.45 GHz
NTTU Throu. (w/ns) *	1024	1024/512	2048	-	768 -
BConvU Throu. (w/ns)	16384	16384/8192	-	-	672 -
EWEU Throu. (w/ns) ‡	2048	2048/1024	1024	7760	512 5461
HBM BW (TB/s)	1	1	1	1.76	1
On-chip BW (TB/s) †	36+36	72+72	-	140	36 96
On-chip Cap (MB) †	180+18	281	-	-	44/84 -
Optimization for PKB Dataflow Mode	Min-KS EVF	Hybrid EVF	Hoisting EVF	Hoisting IRF	Hoisting/Hybrid IRF/Hybrid
Our FHE parameters	$N = 2^{16}, L = 35, L_{\text{eff}} = 8, k = 12, \alpha = 12, \text{dnum} = 3, \lambda = 128\text{-bit}$				

* Since NTTUs in HE² are dynamically allocated between NTT and INTT, the throughput is averaged over all NTT and INTT in the keyswitch with different levels. Throughput is measured in words per nanosecond (w/ns).

‡ The throughput of EWEU is defined as that of IPs.

† SRAM/register bandwidth for ASICs or xPUs, and near-memory bandwidth for xMUs.

‡ SRAM/register capacity for ASICs or xPUs.

fusion to eliminate the row-switch overhead for intermediate results during sequential MemOps such as IP and PMul shown in Fig. 10(d).

In addition, the automorphism is implemented entirely within DRAM without extra hardware. By leveraging the shared multi-level controllers and local buffers—an approach inspired by [51]—data movement is performed hierarchically: via the global row buffer (2048 coeff/cycle) for intra-bank transfers, the bank I/O controller (128 coeff/cycle) for inter-bank transfers, and the GBus controller (32 coeff/cycle) for inter-bank-group transfers. This reuse of native DRAM data paths eliminates the need for new logic while achieving a 1.10 $\times$ speedup over the two-level automorphism in F1 [43], making it an efficient yet low-overhead complement to the xMU design.

Our xMU adopts HBM with 8 GB capacity and 1 TB/s bandwidth to balance computation and communication demands. This configuration ensures sufficient throughput for heterogeneous data exchange and supports the PKB fusion in HERO. Further sensitivity analysis is presented in Sec. VII-E.

VI. METHODOLOGY

A. HE² Evaluation

We choose SHARP [25] as the core baseline to evaluate different algorithms and architectural variants, since SHARP achieves high performance under low hardware overhead through short word-length design and efficient on-chip memory management. SHARP has demonstrated state-of-the-art energy-delay-area product (EDAP) in existing ASIC implementations [16], [26], which aligns with our motivation to investigate heterogeneous architectures that balance computational efficiency and hardware cost.

Simulator. We develop cycle-accurate performance simulators to evaluate: (1) **SHARP** by strictly following its technical specifications, replicating its architecture and Min-KS-based dataflow strategy. In our end-to-end benchmarks, our simulated performance differs from the results reported in the original paper by 1.20% on average. We further model SHARP

TABLE III
AREA AND PEAK POWER BREAKDOWN.

Modules	Area (mm ²)	Power (W)
96 $\times$ NTTU	2.05	8.71
672 $\times$ BConvU	5.32	22.6
OF-Twist	0.12	0.53
EWEU	0.67	2.84
Scratchpad-SM (44 MB)	9.60	7.90
Scratchpad-LM (84 MB)	17.9	13.1
NoC	0.01	0.03
HBM PHY	29.6	31.9
xPU-SM (7 nm)	47.4	74.5
xPU-LM (7 nm)	55.7	79.7
Comp. Units	11.9	11.0
Registers	0.40	0.75
Total xMU (12 nm)	12.2	11.8
HE²-SM (1 xPU 2 xMU)	71.9	98.0
HE²-LM (1 xPU 2 xMU)	80.2	103
BTS (7 nm)	374	163
CLake (7 nm)	223	320
ARK (7 nm)	418	281
SHARP (7 nm)	179	-
UFC (7 nm)	198	-
FAST (7 nm)	284	338
FHENDI (12 nm)	890	628

with hoisting by prefetching the required *evks* before each hoisted PKB, thereby providing a convincing approximation of SHARP’s actual performance with hoisting. (2) **HE²-LM**, with a larger 84 MB on-chip memory supporting hybrid dataflow. (3) **HE²-SM**, a smaller variant with 44 MB on-chip memory, sufficient for ciphertexts but unable to store any *evk*, thus only supporting the IRF. (4) **SHARP-xMU**, a heterogeneous hardware combining SHARP with NMP-enabled HBM, adopting IRF dataflow.

Hardware and Algorithmic Specifications. A comparison of the modeling configurations between HE² and prior works is presented in Table II. Compared with ASIC-based designs [16], [25], HE² xPU features lower module parallelism and hardware overhead yet delivers superior performance under limited resources. This improvement stems from HERO (Sec. IV), which exploits the hoisting potential to reduce ComOp burden for xPU, and the pipelined xPU design (Sec. V-B), which enables inter-operator overlap. Compared with NMP-based works [40], we adopt a lightweight near-memory integration: only simple SIMD-style MemOps are offloaded to the xMU, and a fusion strategy (Sec. V-E) ensures high near-memory bandwidth utilization similar to [45] but with minimal hardware overhead. We adopt the same algorithmic parameters as SHARP, as shown in Table II. We use an FFT-like bootstrapping [6] with three stages. Min-KS [26] is employed to reduce *evk* storage when neither hoisting nor HERO is applied.

BSGS Configurations. In our evaluations of bootstrapping, HELR, and ResNet, we disable BSGS in both C2S and S2C under the 8 GB HBM capacity constraint. For BERT, BSGS is still adopted in the first FFT stage of C2S during its bootstrapping, with a baby step of 2 and a giant step of 32. This is because the first FFT stage operates at a relatively high ciphertext level. If BSGS is disabled in this stage, the resulting *evk* would surpass the available HBM capacity.

TABLE IV
END-TO-END LATENCY (ms), EDP (J-ms), EDAP (J-ms-mm²) EVALUATION.

	Bootstrapping [6]			HELRL [21]			ResNet-20 [30]			ResNet-56 [30]		
	Latency	EDP	EDAP	Latency	EDP	EDAP	Latency	EDP	EDAP	Latency	EDP	EDAP
Anaheim (GPU-NMP)	29.3	-	-	41.2	-	-	1020	-	-	3476	-	-
BTS (ASIC)	22.9	-	-	28.4	-	-	1910	-	-	6509	-	-
CLake (ASIC)	6.32	9.91	4.68K	15.2	-	-	321	28.7K	13.6M	1094	333K	158M
ARK (ASIC)	3.52	1.67	699	7.42	5.54	2.32K	125	2.03K	848K	426	23.6K	9.85M
SHARP (ASIC)	3.12	0.94	168	2.53	2.56	458	99	648	116K	337	7.51K	1.34M
UFC (ASIC)	2.60	0.45	89.8	2.11	1.15	229	90	331	65.5K	302	3.73K	738K
FAST (ASIC)	1.38	0.20	56.8	1.33	2.20	625	61	595	169K	205	6.72K	1.91M
FHENDI (NMP)	1.56	-	-	-	-	-	83	-	-	284	-	-
HE²-SM	1.42	0.16	11.2	1.79	0.87	62.5	69.7	234	16.8K	232	2.60K	186K
HE²-LM	1.33	0.13	10.7	1.70	0.75	59.9	71.9	219	17.5K	240	2.43K	194K

* ResNet-56 performance of the existing designs is obtained by scaling the reported results on ResNet-20 according to computation load.

B. Benchmarks

Bootstrapping follows state-of-the-art fully-packed implementation [25], [26] with 8 efficient levels. **HELRL** [21] is an ML workload which trains a binary classifier. We use a batch size of 1024 and report the average latency over 32 iterations. We evaluate **ResNet-20** and **ResNet-56** [30], which are CNN models adopting a multiplexed packing method, with encrypted images of size $32 \times 32 \times 3$ and batch size of 1. Furthermore, we evaluate a 12-layer **BERT-based** model [53] for a single inference with a 128×768 input sequence.

VII. EVALUATION

A. Implementation of HE²

We implement HE² in RTL and estimate its power and area. We employ the 7 nm TSMC PDK for the xPU and the TSMC 12 nm PDK for the xMU PEs. We integrate the OF-Twist proposed in ARK [26] for twiddle factor generation, and implement the KSKGen proposed in [44] for *evk* generation in xMU PEs. The EWU is implemented following SHARP [25], with four modular multipliers and two modular adders per unit, but only with one-quarter of SHARP's parallelism, dedicated to EWUs on the xPU. On-chip wiring and scratchpad memory are modeled using CACTI-6.0 [38]. Our xMU PEs are integrated within two HBM2 stacks [46], [48], aligning with the off-chip memory in prior ASIC-based designs [12], [25], [26], which provides a total of 1 TB/s off-chip bandwidth, and 8 GB capacity. The xPU operates at 1 GHz, while PEs within the xMU run at 450 MHz.

Table III shows the area and power comparison with existing works [25], [26], [29], [40], [44]. On the one hand, by reducing both the large on-chip memory and per-operator parallelism, we significantly lower the area and power overheads of the xPU. On the other hand, the xMU PEs adopt a feasible bank-level design consistent with prior near-memory architectures [13], [32], [33], maintaining peak power within the HBM's all-bank-interleave access budget [39], [47] and operating within the 85 °C thermal envelope. Our xMU area evaluation considers the logical integration density scaling factor in DRAM technologies according to [34], [56]. The xMU PEs occupy only 11.1% of the HBM module area, satisfy RTL-verified timing constraints, and preserve HBM bank

I/O compatibility under standard design rules [47], thereby demonstrating the feasibility for implementation [20], [47].

B. End-to-end Evaluation and Ablation Study

We evaluate HE² on four benchmarks and compare the performance against a GPU-NMP heterogeneous design (Anaheim [27]), ASIC (BTS [29], CLake [44], ARK [26], SHARP [25], UFC [55], and FAST [16]), and NMP (FHENDI [40]) accelerators, as shown in the Table IV. Overall, HE² achieves the optimal EDP and EDAP, and state-of-the-art latency performance. **(1)** Compared with Anaheim [27], which is a GPU-NMP heterogeneous design applying hoisting on raw CKKS programs, HE² achieves an average $22.0\times$ improvement in latency. This is because: *a)* Executing FHE operators on a general-purpose GPU architecture makes it difficult to achieve efficient overlap between operators. *b)* The lack of a customized keyswitch dataflow design in Anaheim prevents computation communication overlap, failing to address the costly communication on the critical path. *c)* HERO fuses PKBs in the program rather than directly applying hoisting, achieving much higher keyswitch parallelism than the original CKKS program used in Anaheim. **(2)** FAST [16] is the fastest prior ASIC accelerator, leveraging an advanced keyswitch algorithm [28] at the cost of considerable compute resources and on-chip storage. As shown in Table III, FAST's area and peak power are $3.54\times$ and $3.28\times$ those of HE²-LM, respectively. Thus, HE² delivers $2.49\times$ and $8.81\times$ improvements in EDP and EDAP over FAST, while achieving comparable latency. In addition, **(3)** compared to the state-of-the-art NMP accelerator FHENDI [40], HE² demonstrates similar end-to-end performance with $11.1\times$ reduction in area and $6.10\times$ reduction in peak power. Overall, our performance improvement can be attributed to two factors: **(1) the co-design of DFG optimizations and heterogeneous dataflow**, and **(2) architectural optimizations**.

To provide deeper insights, we carry out an ablation study based on the SHARP baseline, which is selected for its state-of-the-art EDAP and hardware efficiency, as shown in Fig. 14. For SHARP with the EVF dataflow (1st and 2nd column), we apply two algorithmic optimizations, Min-KS and hoisting. The results show that the low *evk* reuse introduced by

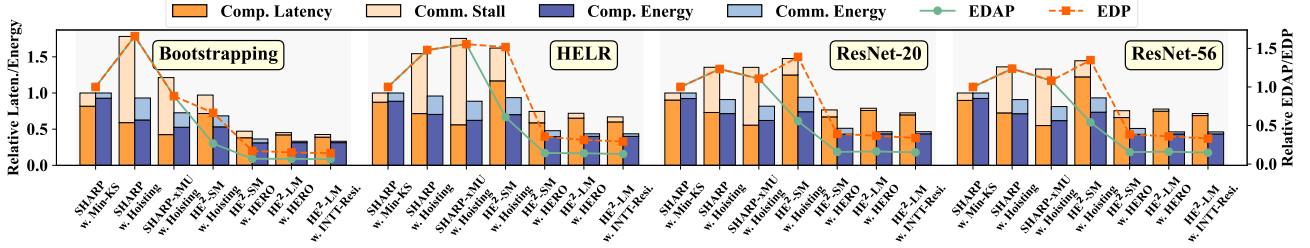


Fig. 14. Ablation study results. SHARP w. Hoisting exhibits inadaptability to hoisting due to long memory stalls (2nd column). SHARP-xMU is a heterogeneous design adopting IRF dataflow, with its xPU aligned with SHARP, which shows partial hoisting compatibility (3rd column). HE²-SM, using the same IRF dataflow, achieves similar performance to SHARP-xMU but with fewer stalls (4th column), as its dual-pipelined xPU hides communication. And HERO (5th column) shows significant computation and communication reduction compared to hoisting. HE²-LM adopts a hybrid dataflow (6th column), further mitigating communication where hoisting is inapplicable. The INTT-Resident (7th column) strategy enhances xPU computation parallelism.

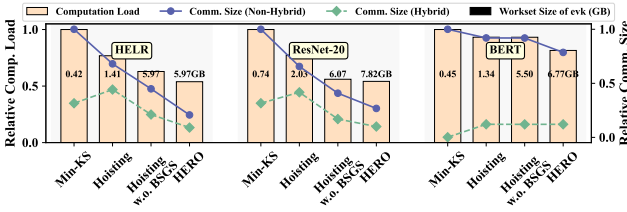


Fig. 15. Comparison of algorithmic optimizations effects on HE². HERO enhances computational and communication efficiency at the cost of increased evk storage. Compared with applying hoisting in the original program or in the program with BSGS disabled (Hoisting w.o. BSGS), HERO achieves a greater reduction in both computation and communication.

hoisting increases off-chip memory access under EVF, causing performance degradation compared with Min-KS.

In contrast, a heterogeneous architecture with IRF dataflow (3rd column) shifts from reusing *evk* on the xPU to reusing intermediate results across the *evks* stored on the xMU, thereby partially reducing the communication overhead. However, low parallelism PKBs gain little from hoisting. In such cases, IRF leads to intermediate ciphertext movement that surpasses the cost of *evk* loading, and communication on the critical path, resulting in significant stalls (e.g., the 3rd column of HELK).

To address communication stalls and provide more flexible dataflow support, **first**, the HE² dual-level overlapping xPU design hides most communication latency while maintaining performance (4th column); **second**, increased PKB parallelism in HERO amplifies hoisting benefits for both computation and communication (5th column); **third**, hybridizing IRF and EVF with an additional on-xPU buffer sufficient for one *evk* further reduces communication stalls (6th column). Specifically, for PKBs with parallel keyswitches, hoisting and IRF reuse intermediate results to minimize data movement, whereas for PKBs with a single keyswitch, EVF preloads the required *evk* on xPU, since the overhead of preloading one *evk* is smaller than that of moving intermediate results. **Finally**, the INTT-Resident mechanism enhances xPU’s computation parallelism (7th column). In the hybrid dataflow, the IRF region is bounded by communication, while the EVF region is limited by computation latency. Therefore, the INTT-Resident

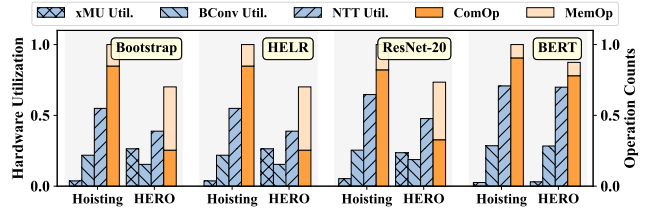


Fig. 16. Hardware utilization and operation counts of HE². HERO increases the proportion of MemOps and the utilization of xMU. Both hoisting and HERO are evaluated under an IRF dataflow.

optimization mainly benefits the EVF parts. In summary, HE²-LM achieves a $9.23\times$ EDAP and $4.13\times$ EDP improvement over SHARP. Moreover, communication stalls account for only 6.67% of the total execution time, and communication energy overhead is reduced to 6.60%.

C. Impact of HERO on Computation and Communication

We evaluate the algorithmic optimizations of HERO. As shown in Fig. 15, hoisting merges ModUps and ModDowns among parallel keyswitches to cut communication and computation (2nd column). When BSGS is disabled (3rd column), or when BERT adopts a large gap between the baby-step and giant-step parameters (see Sec. VI-A for details), more parallel keyswitches occur in bootstrapping, yielding further gains. Finally, HERO consolidates the low-parallelism PKBs across the program (4th column), achieving the optimal overall performance.

Furthermore, the dataflow strategy strongly influences the communication volume. IRF markedly reduces communication for PKBs with high parallelism, whereas for PKBs with low parallelism, the communication cost of loading a single *evk* is smaller than that of transferring intermediate results in IRF, making EVF more efficient. As a result, the hybrid strategy, which selectively applies IRF and EVF, achieves lower communication overhead than pure IRF.

D. Hardware Utilization Analysis

We evaluate the hardware utilization, as shown in Fig. 16. When HE² adopts the IRF dataflow with hoisting (1st column), the overall execution becomes bounded by xPU computation

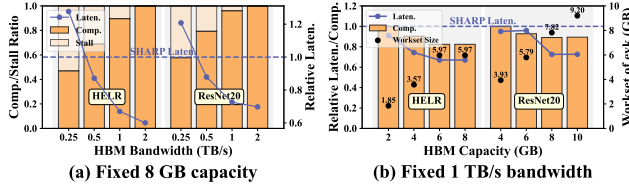


Fig. 17. Performance scaling of HE^2 with respect to HBM bandwidth and capacity shows a gradual saturation trend, demonstrating the effectiveness of communication optimization in mitigating bandwidth requirements. And the optimal PKB fusion solution found by HERO can be achieved within an 8 GB memory budget. SHARP adopts an HBM with 1 TB/s bandwidth and 8 GB capacity.

rather than memory, in contrast to SHARP [25], where hoisting exposes memory bottlenecks. This shift occurs because IRF eliminates the need to load *evks*. With HERO enabled (2nd column), the range and efficiency of hoisting are further enhanced, leading to a higher proportion of MemOp workloads and increased xMU utilization.

E. Sensitivity

We investigate HE^2 's sensitivity to variations in the NMP-enabled HBM (xMU) bandwidth and capacity. As shown in Fig. 17(a), increasing the bandwidth reduces communication stalls, leading to performance improvement. Results show that **HE^2 outperforms SHARP at the same 1 TB/s bandwidth and maintains comparable performance to SHARP with bandwidth at least 0.5 TB/s.**

The experiment shown in Fig. 17(b) evaluates the effectiveness of HERO's PKB fusion strategy under different capacity constraints. With the same capacity of 8 GB, HE^2 achieves higher performance than SHARP. Furthermore, **8 GB is sufficient to identify the optimal fusion strategies** for the benchmarks tested. Although a larger memory capacity enables more aggressive fusion strategies, the PKBs at this point already exhibit substantial parallelism. Further fusion results in an explosive increase in IP computation, which in turn degrades performance.

VIII. RELATED WORK

ASIC- and FPGA-based FHE accelerators. HEAX [41] and Roy et al. [42] proposed FPGA accelerators for CKKS, but without support for the bootstrapping. Poseidon [52] and FAB [2] further enable arbitrary-depth CKKS computation with bootstrapping, but their acceleration performance remains lower compared to ASIC implementations. The first CKKS accelerator F1 [43] only supports shallow benchmarks. Craterlake [44], BTS [29], and ARK [26] designed highly parallel modules combined with large on-chip memory to support deep CKKS applications and achieve high acceleration performance. SHARP [25] further identified 36-bit datapaths as the most efficient choice, which reduces the hardware overhead. Alchemist [36], Trinity [12], and UFC [55] further proposed more general-purpose designs that support both CKKS and TFHE schemes. FAST [16] employs a more

advanced keyswitch algorithm, which significantly reduces computation cost and enhances acceleration performance.

NMP-based FHE accelerators. Gupta et al. [17] proposed a DRAM-based NMP solution based on UPMEM [50] for BFV scheme acceleration. MemFHE [18] and FHE-PIM [19] leverage an RRAM-based NMP architecture for FHEW scheme acceleration. These designs cannot be extended to CKKS with more complex primitives. FHENDI [40] adopted an HBM-based design, where massively parallel NMP units and parallel bootstrapping enable high performance, but its large area and power overhead limit manufacturability. FlexMem [45] revealed the low near-memory bandwidth utilization but used a complex in-memory network restricted by the available metal layers in DRAM technology [49].

Hoisting-based works. Anaheim [27] applied hoisting to the raw CKKS program and identified the bottleneck in MemOps, and proposed a heterogeneous GPU-NMP architecture. However, its hoisting effectiveness is limited by low keyswitch parallelism in the native program. Moreover, without a customized xPU design or dataflow optimization for computation-communication balance, the overall acceleration remains limited. FAST [16] applied hoisting at low ciphertext levels without restructuring the program to better leverage it. Orion [15] represented convolutions as plaintext-ciphertext matrix multiplications to exploit BSGS and hoisting. Nevertheless, Orion ignored the effect of BSGS on the efficiency of hoisting and exhibited lower generality compared with HERO.

IX. CONCLUSION

We present the first ASIC-NMP heterogeneous CKKS accelerator HE^2 , which, for the first time, addresses the long-standing dilemma between high hardware cost and acceleration performance commonly encountered in both ASIC and NMP monolithic designs. To tackle the xPU-xMU communication bottleneck caused by the complex data dependencies between operators, we introduce algorithmic optimizations and microarchitectural enhancements, which effectively reduce communication frequency and hide latency. HE^2 achieves $1.66\times$ and $1.17\times$ speedups over state-of-the-art ASIC and NMP accelerators, as well as $2.49\times$ and $12.4\times$ reductions in area, thereby highlighting a practical design avenue for future FHE accelerators.

ACKNOWLEDGMENTS

We thank the anonymous reviewers and shepherd for their insightful comments and suggestions. This work is partially supported by the NSF of China (Grants No.62341411, 62222214), Strategic Priority Research Program of the Chinese Academy of Sciences (Grants No.XDB0660200, XDB0660201, XDB0660202), and Youth Innovation Promotion Association CAS.

REFERENCES

- [1] R. Agrawal, L. De Castro, C. Juvekar, A. Chandrakasan, V. Vaikuntanathan, and A. Joshi, "Mad: Memory-aware design techniques for accelerating fully homomorphic encryption," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023, pp. 685–697.

- [2] R. Agrawal, L. de Castro, G. Yang, C. Juvekar, R. Yazicigil, A. Chandrakasan, V. Vaikuntanathan, and A. Joshi, "Fab: An fpga-based accelerator for bootstrappable fully homomorphic encryption," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2023, pp. 882–895.
- [3] J.-C. Bajard, J. Eynard, M. A. Hasan, and V. Zucca, "A full rns variant of fv like somewhat homomorphic encryption schemes," in *International Conference on Selected Areas in Cryptography*. Springer, 2016, pp. 423–442.
- [4] J.-P. Bossuat, C. Mouchet, J. Troncoso-Pastoriza, and J.-P. Hubaux, "Efficient bootstrapping for approximate homomorphic encryption with non-sparse keys," in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 2021, pp. 587–617.
- [5] A. Brutzkus, R. Gilad-Bachrach, and O. Elisha, "Low latency privacy preserving inference," in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, K. Chaudhuri and R. Salakhutdinov, Eds., vol. 97. PMLR, 09–15 Jun 2019, pp. 812–821.
- [6] H. Chen, I. Chillotti, and Y. Song, "Improved bootstrapping for approximate homomorphic encryption," in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 2019, pp. 34–54.
- [7] J. H. Cheon, K. Han, A. Kim, M. Kim, and Y. Song, "A full rns variant of approximate homomorphic encryption," in *International Conference on Selected Areas in Cryptography*. Springer, 2018, pp. 347–368.
- [8] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène, "Tfhe: fast fully homomorphic encryption over the torus," *Journal of Cryptology*, vol. 33, no. 1, pp. 34–91, 2020.
- [9] L. T. Clark, V. Vashishtha, L. Shifren, A. Gujja, S. Sinha, B. Cline, C. Ramamurthy, and G. Yeric, "Asap7: A 7-nm finfet predictive process design kit," *Microelectronics Journal*, vol. 53, pp. 105–115, 2016. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S002626921630026X>
- [10] R. Dathathri, B. Kostova, O. Saarikivi, W. Dai, K. Laine, and M. Musuvathi, "Eva: An encrypted vector arithmetic language and compiler for efficient homomorphic computation," in *Proceedings of the 41st ACM SIGPLAN conference on programming language design and implementation*, 2020, pp. 546–561.
- [11] R. Dathathri, O. Saarikivi, H. Chen, K. Laine, K. Lauter, S. Maleki, M. Musuvathi, and T. Mytkowicz, "Chet: an optimizing compiler for fully-homomorphic neural-network inferencing," in *Proceedings of the 40th ACM SIGPLAN conference on programming language design and implementation*, 2019, pp. 142–156.
- [12] X. Deng, S. Fan, Z. Hu, Z. Tian, Z. Yang, J. Yu, D. Cao, D. Meng, R. Hou, M. Li, L. Qian, and Z. Mingzhe, "Trinity: A general purpose the accelerator," in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2024, pp. 338–351.
- [13] F. Devaux, "The true processing in memory accelerator," in *2019 IEEE Hot Chips 31 Symposium (HCS)*. IEEE, 2019, pp. 1–24.
- [14] L. Ding, S. Bian, P. He, Y. Xu, G. Qu, and J. Zhang, "Apache: A processing-near-memory architecture for multi-scheme fully homomorphic encryption," *arXiv preprint arXiv:2404.15819*, 2024.
- [15] A. Ebel, K. Garimella, and B. Reagen, "Orion: A fully homomorphic encryption framework for deep learning," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 734–749. [Online]. Available: <https://doi.org/10.1145/3676641.3716008>
- [16] S. Fan, X. Deng, L. Kong, G. Shi, G. Fan, D. Meng, R. Hou, and M. Zhang, "Fast: An the accelerator for scalable-parallelism with tunable-bit," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 92–106.
- [17] H. Gupta, M. Kabra, J. Gómez-Luna, K. Kanellopoulos, and O. Mutlu, "Evaluating homomorphic operations on a real-world processing-in-memory system," in *2023 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 2023, pp. 211–215.
- [18] S. Gupta, R. Cammarota, and T. Šimunić, "Memfhe: End-to-end computing with fully homomorphic encryption in memory," *ACM Trans. Embed. Comput. Syst.*, vol. 23, no. 2, Mar. 2024. [Online]. Available: <https://doi.org/10.1145/3569955>
- [19] S. Gupta and T. v. Rosing, "Invited: Accelerating fully homomorphic encryption with processing in memory," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*, 2021, pp. 1335–1338.
- [20] J.-H. Han, R. E. West, K. Torres-Castro, N. Swami, S. Khan, and M. Stan, "Power and thermal modeling of in-3d-memory computing," in *2021 International Symposium on Devices, Circuits and Systems (ISDCS)*. IEEE, 2021, pp. 1–4.
- [21] K. Han, S. Hong, J. H. Cheon, and D. Park, "Logistic regression on homomorphic encrypted data at scale," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, no. 01, pp. 9466–9471, Jul. 2019. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/5000>
- [22] K. Han and D. Ki, "Better bootstrapping for approximate homomorphic encryption," in *Topics in Cryptology – CT-RSA 2020*, S. Jarecki, Ed. Cham: Springer International Publishing, 2020, pp. 364–390.
- [23] M. He, C. Song, I. Kim, C. Jeong, S. Kim, I. Park, M. Thottethodi, and T. Vijaykumar, "Newton: A dram-maker's accelerator-in-memory (aim) architecture for machine learning," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 372–385.
- [24] A. Kim, Y. Polyakov, and V. Zucca, "Revisiting homomorphic encryption schemes for finite fields," in *Advances in Cryptology–ASIACRYPT 2021: 27th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 6–10, 2021, Proceedings, Part III 27*. Springer, 2021, pp. 608–639.
- [25] J. Kim, S. Kim, J. Choi, J. Park, D. Kim, and J. H. Ahn, "Sharp: A short-word hierarchical accelerator for robust and practical fully homomorphic encryption," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–15.
- [26] J. Kim, G. Lee, S. Kim, G. Sohn, M. Rhu, J. Kim, and J. H. Ahn, "Ark: Fully homomorphic encryption accelerator with runtime data generation and inter-operation key reuse," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2022, pp. 1237–1254.
- [27] J. Kim, S. Yun, H. Ji, W. Choi, S. Kim, and J. H. Ahn, "Anaheim: Architecture and algorithms for processing fully homomorphic encryption in memory," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2025, pp. 1158–1173.
- [28] M. Kim, D. Lee, J. Seo, and Y. Song, "Accelerating he operations from key decomposition technique," in *Annual International Cryptology Conference*. Springer, 2023, pp. 70–92.
- [29] S. Kim, J. Kim, M. J. Kim, W. Jung, J. Kim, M. Rhu, and J. H. Ahn, "Bts: An accelerator for bootstrappable fully homomorphic encryption," in *Proceedings of the 49th annual international symposium on computer architecture*, 2022, pp. 711–725.
- [30] E. Lee, J.-W. Lee, J. Lee, Y.-S. Kim, Y. Kim, J.-S. No, and W. Choi, "Low-complexity deep convolutional neural networks on fully homomorphic encryption using multiplexed parallel convolutions," in *Proceedings of the 39th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvari, G. Niu, and S. Sabato, Eds., vol. 162. PMLR, 17–23 Jul 2022, pp. 12 403–12 422.
- [31] J.-W. Lee, H. Kang, Y. Lee, W. Choi, J. Eom, M. Deryabin, E. Lee, J. Lee, D. Yoo, Y.-S. Kim, and J.-S. No, "Privacy-preserving machine learning with fully homomorphic encryption for deep neural network," *IEEE Access*, vol. 10, pp. 30 039–30 054, 2022.
- [32] S. Lee, K. Kim, S. Oh, J. Park, G. Hong, D. Ka, K. Hwang, J. Park, K. Kang, J. Kim *et al.*, "A 1nm 1.25 v 8gb, 16gb/s/pin gddr6-based accelerator-in-memory supporting 1tflops mac operation and various activation functions for deep-learning applications," in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65. IEEE, 2022, pp. 1–3.
- [33] S. Lee, S.-h. Kang, J. Lee, H. Kim, E. Lee, S. Seo, H. Yoon, S. Lee, K. Lim, H. Shin *et al.*, "Hardware architecture and software stack for pim based on commercial dram technology: Industrial product," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021, pp. 43–56.
- [34] H. Liu, L. Zheng, Y. Huang, C. Liu, X. Ye, J. Yuan, X. Liao, H. Jin, and J. Xue, "Accelerating personalized recommendation with cross-level near-memory processing," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–13.
- [35] Y. Liu, J. Lai, L. Li, T. Sui, L. Xiao, P. Yuan, X. Zhang, Q. Zhu, W. Chen, and J. Xue, "Resbm: Region-based scale and minimal-level bootstrapping management for the via min-cut," in *Proceedings of the 30th ACM International Conference on Architectural Support for*

- Programming Languages and Operating Systems, Volume 1*, 2025, pp. 924–939.
- [36] J. Mu, H. Han, S. Shi, J. Ye, Z. Liu, S. Liang, M. Li, M. Zhang, S. Bian, X. Hu, H. Li, and X. Li, “Alchemist: A unified accelerator architecture for cross-scheme fully homomorphic encryption,” in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, ser. DAC ’24. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: <https://doi.org/10.1145/3649329.3657331>
 - [37] J. Mu, Y. Ren, W. Wang, Y. Hu, S. Chen, C.-H. Chang, J. Fan, J. Ye, Y. Cao, H. Li *et al.*, “Scalable and conflict-free ntt hardware accelerator design: Methodology, proof, and implementation,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 5, pp. 1504–1517, 2022.
 - [38] N. Muralimanohar, R. Balasubramonian, and N. P. Jouppi, “Cacti 6.0: A tool to model large caches,” *HP laboratories*, vol. 27, p. 28, 2009.
 - [39] J. Park, J. Choi, K. Kyung, M. J. Kim, Y. Kwon, N. S. Kim, and J. H. Ahn, “Attace! unleashing the power of pim for batched transformer-based generative model inference,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2024, pp. 103–119.
 - [40] Y. Park, A. Amarnath, S. Pal, K. Swaminathan, A. Buyuktosunoglu, H. Shaul, E. Aharoni, N. Drucker, W. D. Lu, O. Soceanu *et al.*, “Fhendi: A near-dram accelerator for compiler-generated fully homomorphic encryption applications,” in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2025, pp. 1127–1142.
 - [41] M. S. Riazzi, K. Laine, B. Pelton, and W. Dai, “Heax: An architecture for computing on encrypted data,” in *Proceedings of the twenty-fifth international conference on architectural support for programming languages and operating systems*, 2020, pp. 1295–1309.
 - [42] S. S. Roy, F. Turan, K. Jarvinen, F. Vercauteren, and I. Verbauwhede, “Fpga-based high-performance parallel architecture for homomorphic computing on encrypted data,” in *2019 IEEE International symposium on high performance computer architecture (HPCA)*. IEEE, 2019, pp. 387–398.
 - [43] N. Samardzic, A. Feldmann, A. Krastev, S. Devadas, R. Dreslinski, C. Peikert, and D. Sanchez, “F1: A fast and programmable accelerator for fully homomorphic encryption,” in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 238–252.
 - [44] N. Samardzic, A. Feldmann, A. Krastev, N. Manohar, N. Genise, S. Devadas, K. Eldefrawy, C. Peikert, and D. Sanchez, “Craterlake: a hardware accelerator for efficient unbounded computation on encrypted data,” in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022, pp. 173–187.
 - [45] S. Shi, H. Han, J. Mu, X. Zheng, L. Liang, H. Lu, Z. Du, X. Li, and X. Hu, “Flexmem: High-parallel near-memory architecture for flexible dataflow in fully homomorphic encryption,” in *2026 31st Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 2026, pp. 1174–1180.
 - [46] K. Sohn, W.-J. Yun, R. Oh, C.-S. Oh, S.-Y. Seo, M.-S. Park, D.-H. Shin, W.-C. Jung, S.-H. Shin, J.-M. Ryu *et al.*, “A 1.2 v 20 nm 307 gb/s hbm dram with at-speed wafer-level io test scheme and adaptive refresh considering temperature distribution,” *IEEE Journal of Solid-State Circuits*, vol. 52, no. 1, pp. 250–260, 2016.
 - [47] D. S. Specification, “Jedec standard,” 2009.
 - [48] J. Standard, “High bandwidth memory (hbm) dram,” *Jesd235*, vol. 16, 2013.
 - [49] B. Tian, Y. Li, L. Jiang, S. Cai, and M. Gao, “Ndpbridge: Enabling cross-bank coordination in near-dram-bank processing architectures,” in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 628–643.
 - [50] UPMEM. (2025) Upmem official website. [Online]. Available: <https://www.upmem.com/>
 - [51] Y. Wang, L. Orosa, X. Peng, Y. Guo, S. Ghose, M. Patel, J. S. Kim, J. G. Luna, M. Sadrosadati, N. M. Ghiasi *et al.*, “Figaro: Improving system performance via fine-grained in-dram data relocation and caching,” in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 313–328.
 - [52] Y. Yang, H. Zhang, S. Fan, H. Lu, M. Zhang, and X. Li, “Poseidon: Practical homomorphic encryption accelerator,” in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 870–881.
 - [53] J. Zhang, X. Yang, L. He, K. Chen, W.-j. Lu, Y. Wang, X. Hou, J. Liu, K. Ren, and X. Yang, “Secure transformer inference made non-interactive,” *Cryptology ePrint Archive*, 2024.
 - [54] M. Zhou, Y. Nam, P. Gangwar, W. Xu, A. Dutta, C. Wilkerson, R. Cammarota, S. Gupta, and T. Rosing, “Fhemem: A processing in-memory accelerator for fully homomorphic encryption,” *IEEE Transactions on Emerging Topics in Computing*, 2025.
 - [55] M. Zhou, Y. Nam, X. Wang, Y. Lee, C. Wilkerson, R. Kumar, S. Taneja, S. Mathew, R. Cammarota, and T. Rosing, “Ufc: A unified accelerator for fully homomorphic encryption,” in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2024, pp. 352–365.
 - [56] M. Zhou, W. Xu, J. Kang, and T. Rosing, “Transpim: A memory-based acceleration via software-hardware co-design for transformer,” in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2022, pp. 1071–1085.